

07-00

Chapter Overview — DynamoDB

Amazon DynamoDB is a fully managed NoSQL database that delivers single-digit millisecond performance at any scale. Unlike Aurora (which stores data in tables with rows and columns and requires SQL), DynamoDB stores data as items with flexible attributes and is accessed via simple key lookups. ShopFlow uses DynamoDB for: shopping cart (needs instant reads/writes for active sessions), order history (10M+ records, frequent user lookups), and product inventory counts (thousands of updates per second during flash sales).

■ When to Use DynamoDB vs Aurora

Aurora is better when: you need complex queries with JOINS, transactions across multiple tables, ad-hoc reporting, or strict relational integrity. DynamoDB is better when: you need sub-10ms reads at any scale, simple key-based lookups, flexible schema (different items can have different attributes), or serverless architecture (Lambda + DynamoDB = zero infrastructure management).

Example: ShopFlow shopping cart: Aurora is wrong choice — cart is accessed on every page load (high read rate), has flexible structure (each cart is different), and does not need JOINS. DynamoDB is right: user_id lookup returns entire cart in 1-3ms regardless of whether 100 or 10 million users are shopping simultaneously.

Feature	Aurora PostgreSQL	DynamoDB
Data model	Relational: tables, rows, columns, foreign keys	Key-value/document: items with flexible attributes; no schema required
Query language	SQL: SELECT, JOIN, GROUP BY, complex filters	API: GetItem, Query, Scan, PutItem — limited to key-based access
Scaling	Vertical (bigger instance) + read replicas	Horizontal: auto-scales to millions of requests/second; no instance management
Performance	5-50ms typical; degrades without proper indexing	1-3ms guaranteed at any scale with proper key design
Transactions	Full ACID transactions across all tables	ACID transactions supported within a single table (up to 25 items)
Cost model	Per-hour instance + storage	Per read/write unit consumed; On-Demand scales to zero cost when idle

07-01

DynamoDB Data Model — Tables, Items, and Attributes

A DynamoDB table has a primary key (mandatory) and optional sort key. Items in the same table can have completely different attributes — there is no schema enforcement. The primary key is the only required attribute. Everything else is flexible. ShopFlow cart table: each item has user_id (partition key), product_id (sort key), quantity, price, added_at — but some items also have discount_code, bundle_id, gift_wrap_requested.

DynamoDB Concept	Definition	ShopFlow Cart Example
Table	A collection of items (like a table in SQL, but with no schema enforcement)	shopflow_carts stores all active shopping carts for all users
Item	A single record in the table (like a row in SQL, but with flexible attributes)	One item is a cart for user 123, product iphone-15, qty 2
Partition Key (PK)	The primary identifier; determines which server stores the item	uses the 'user-123' as the unique cart identifier and key
Sort Key (SK)	Secondary part of primary key; items with same PK are sorted by SK	PK is 'user-123', SK is 'product-iphone-15-pro' — enables: get all items in cart, ordered by price
Attribute	A name-value pair on an item; any item can have any attributes	quantity: 2, price: 999.99 added_at: "2024-11-15T14:30:00Z" gift_wrap_requested: true
Global Secondary Index (GSI)	An alternate key that lets you query items by non-PK attributes	Example GSI: "which users have product iphone-15 in their cart"

Creating a DynamoDB table takes under a minute. The most important decision is the primary key design — you cannot change the primary key after table creation without recreating the table. Think carefully about your access patterns before creating tables.

■ Create the ShopFlow Cart Table

1. Search "DynamoDB" in the console → Tables → Create table
2. Table name: shopflow-cart
3. Partition key: user_id (String) — identifies which user owns each cart item
4. Sort key: product_id (String) — identifies the specific product in the cart
5. Table settings: Customize settings
6. Capacity mode: On-Demand (pay per request; no capacity planning needed for variable traffic)
7. Encryption: AWS owned key (default; free; sufficient for cart data)
8. Click Create table — table is ready in seconds

■ aws > DynamoDB > Tables > Create table

Create table

Table name

■ Use lowercase with hyphens; environment prefix: shopflow-cart-prod

■ Table class

Table class

■ DynamoDB Standard (default; most workloads) ■ DynamoDB Standard-IA (save 60% on storage for infrequently accessed items — not for cart)

■ Primary key

Partition key

■ Type: String — distributes data across DynamoDB partitions

Sort key (optional)

■ Type: String — enables range queries: get all items for a user

■ Capacity mode

Capacity mode

■ On-Demand (recommended for variable traffic; no capacity planning) ■ Provisioned (cheaper for predictable steady-state traffic; requires capacity planning)

Create table

ShopFlow creates all DynamoDB tables in CDK with point-in-time recovery enabled and a TTL attribute for automatic item expiration. The cart table automatically deletes cart items that have not been updated in 7 days (expired sessions), saving storage costs.

```
import * as dynamodb from "aws-cdk-lib/aws-dynamodb";
// Shopping cart table
const cartTable = new dynamodb.Table(this, "CartTable", {
  tableName: "shopflow-cart",
  partitionKey: {name:"user_id", type: dynamodb.AttributeType.STRING},
  sortKey: {name:"product_id", type: dynamodb.AttributeType.STRING},
  billingMode: dynamodb.BillingMode.PAY_PER_REQUEST, // On-Demand
  encryption: dynamodb.TableEncryption.AWS_MANAGED,
  pointInTimeRecovery: true, // free; enables PITR for accidental delete recovery
  timeToLiveAttribute: "expires_at", // auto-delete expired carts after 7 days
  removalPolicy: RemovalPolicy.RETAIN, // never delete production table on cdk destroy
});
// GSI: query by product_id (which users have this product in cart)
cartTable.addGlobalSecondaryIndex({
  indexName: "product-carts-index",
  partitionKey: {name:"product_id", type: dynamodb.AttributeType.STRING},
  projectionType: dynamodb.ProjectionType.ALL,
});
// Order history table (high volume – 24M items/year)
const orderHistoryTable = new dynamodb.Table(this, "OrderHistory", {
  tableName: "shopflow-order-history",
  partitionKey: {name:"user_id", type: dynamodb.AttributeType.STRING},
  sortKey: {name:"order_id", type: dynamodb.AttributeType.STRING},
  billingMode: dynamodb.BillingMode.PAY_PER_REQUEST,
  pointInTimeRecovery: true,
});
```

07-04

DynamoDB API — Reading and Writing in Java

DynamoDB is accessed via the AWS SDK, not SQL. The key operations: PutItem (create/replace), GetItem (read by primary key), UpdateItem (modify specific attributes), DeleteItem (remove), Query (get all items with same partition key), and Scan (read entire table — use sparingly). ShopFlow uses the DynamoDB Enhanced Client for Java which maps DynamoDB items to Java objects.

```
// DynamoDB Enhanced Client – Java 21
@DynamoDbBean
public class CartItem {
  @DynamoDbPartitionKey String userId;
  @DynamoDbSortKey String productId;
  int quantity;
  double price;
  String addedAt;
  long expiresAt; // Unix timestamp; DynamoDB TTL uses this for auto-deletion
}

DynamoDbEnhancedClient enhancedClient = DynamoDbEnhancedClient.create();
DynamoDbTable cartTable = enhancedClient
  .table("shopflow-cart", TableSchema.fromBean(CartItem.class));
// Add item to cart (PutItem)
cartTable.putItem(CartItem.builder()
  .userId("user-123")
  .productId("prod-iphone-15")
  .quantity(1)
  .price(999.99)
  .addedAt(Instant.now().toString())
  .expiresAt(Instant.now().plus(7, DAYS).getEpochSecond()) // auto-expire in 7 days
  .build());
// Get all cart items for a user (Query – partition key only)
```

```
QueryEnhancedRequest request = QueryEnhancedRequest.builder()
    .queryConditional(QueryConditional.keyEqualTo(k -> k.partitionValue("user-123")))
    .build();
List cartItems = cartTable.query(request).items().stream().toList();
```

07-05

DynamoDB Capacity Modes — On-Demand vs Provisioned

DynamoDB offers two pricing models. On-Demand: pay \$0.25 per million reads and \$1.25 per million writes — no capacity planning, scales automatically. Provisioned: reserve capacity units in advance for a lower per-unit price, but you pay even when not using them. On-Demand is right for ShopFlow's cart (spiky, unpredictable); Provisioned + auto-scaling is right for order-history (steady, predictable growth).

	On-Demand	Provisioned + Auto Scaling
How it works	DynamoDB auto-scales capacity; you pay per request	Reserve min/max capacity; auto scaling adjusts within range
Write cost	\$1.25 per million WCU	\$0.00065 per WCU-hour (~\$0.47/WCU/month)
Read cost	\$0.25 per million RCU	\$0.00013 per RCU-hour (~\$0.09/RCU/month)
Best for	Spiky, unpredictable traffic; new tables with unknown traffic	Predictable, steady-state traffic; if you can forecast utilization within range
Throttling	None — absorbs any traffic spike up to double previous capacity	Exceeds capacity (the begins) max capacity, reads/writes are throttled
ShopFlow use	Cart, notifications (spiky)	Order-history, product-inventory (predictable; Provisioned saves ~3x)

07-06

Global Secondary Indexes — Query Flexibility

DynamoDB's limitation: you can only query items by their primary key (partition key + sort key). Global Secondary Indexes (GSIs) add alternate query paths. ShopFlow's order-history table has user_id as the partition key — great for "show my orders" but cannot answer "show all orders placed in the last hour" without scanning the entire table. A GSI on created_at enables time-based queries.

Access Pattern	Without GSI	With GSI
"Get all orders for user X"	Query partition key user_id = X → 1ms	Same: primary table query; GSI not needed for this
"Get all orders placed today (for ops dashboard)"	Scan entire table (millions of items) → expensive	GSI on partition key = date (YYYY-MM-DD), sort key = created_at → fast
"Get all orders with status=PENDING (for processing pipeline)"	Scan entire table filtering on status → expensive	GSI on partition key = status → Query GSI with status=PENDING
"Which users have product X in their cart (for flash sale promotion)"	Scan entire table expensive at scale	GSI on cart table with product_id as partition key → Query

```
// Add order-status GSI to order-history table
orderHistoryTable.addGlobalSecondaryIndex({
  indexName: "status-created-index",
  partitionKey: {name:"status", type: dynamodb.AttributeType.STRING},
  sortKey: {name:"created_at", type: dynamodb.AttributeType.STRING},
  projectionType: dynamodb.ProjectionType.INCLUDE,
  nonKeyAttributes: ["order_id", "user_id", "total_amount"], // only project needed attrs
});
```

■ GSIs have their own read/write capacity separate from the base table. On-Demand: no configuration needed. Provisioned: provision GSI capacity separately — if base table has 100 WCUs but GSI writes are rarely used, you can provision just 10 WCUs for the GSI.

07-07

DynamoDB TTL — Automatic Item Expiration

DynamoDB TTL (Time To Live) automatically deletes items after a specified timestamp — at no cost. ShopFlow uses TTL to: expire abandoned shopping carts (7 days), delete user session tokens (24 hours), and remove temporary order processing locks (5 minutes). TTL deletion typically happens within 48 hours of expiry — not guaranteed to be instant.

■ Enable TTL on the Cart Table

1. DynamoDB → Tables → shopflow-cart → Additional settings tab
2. Time to live (TTL): click Turn on
3. TTL attribute name: expires_at
4. Click Turn on TTL
5. Now: any item with an expires_at attribute (Unix epoch integer) will be deleted automatically after that time
6. To set expiry: `item.expires_at = Math.floor(Date.now() / 1000) + (7 * 24 * 60 * 60)` (7 days from now)
7. Items without expires_at attribute are NOT affected by TTL (permanent items)

TTL Use Case	Attribute Name	TTL Value	Business Logic
Shopping cart	expires_at	Current time + 7 days	Cart items older than 7 days without updates are abandoned
User session tokens	session_ttl	Current time + 24 hours	Sessions expire after 24 hours; TTL cleans up; user must login again
Order processing lock	lock_expires	Current time + 5 minutes	Prevents duplicate order processing; lock auto-released after timeout
Rate limit counters	window_end	End of current minute	Rate limiting counters (API calls per minute per user) auto-reset

07-08

DynamoDB Streams — React to Data Changes

DynamoDB Streams records every change (insert, update, delete) to a table and makes it available as an ordered stream. Lambda can process these changes in near real-time. ShopFlow uses DynamoDB Streams on the order-history table: when an order status changes to "delivered", a Lambda function triggers to send a review request email via SES.

```
// CDK: DynamoDB Stream + Lambda trigger
const orderHistoryTable = new dynamodb.Table(this, "OrderHistory", {
  // ... (same as before) ...
  stream: dynamodb.StreamViewType.NEW_AND_OLD_IMAGES,
  // NEW_AND_OLD_IMAGES: Lambda receives both before and after the change
});
// Lambda function triggered on every order status change
const orderStatusLambda = new lambda.Function(this, "OrderStatusHandler", {
  runtime: lambda.Runtime.JAVA_21,
  handler: "com.shopflow.OrderStatusHandler::handleRequest",
  // ... memory, timeout, role ...
});
// Connect DynamoDB Stream to Lambda
orderStatusLambda.addEventSource(
  new lambdaEventSources.DynamoEventSource(orderHistoryTable, {
    startingPosition: lambda.StartingPosition.LATEST,
    batchSize: 10, // process up to 10 order changes at once
    retryAttempts: 3,
    bisectBatchOnError: true, // if batch fails, split in half to find bad record
  })
);
```

07-09

DynamoDB Transactions — ACID Across Multiple Items

DynamoDB TransactWriteItems provides ACID transactions across up to 25 items within a single AWS region. ShopFlow uses transactions when placing an order: atomically decrement inventory count AND create the order record AND delete the cart items. Either all three operations succeed or none of them do — no partial state.

```
// Java 21: DynamoDB transaction – place order atomically
```

```
TransactWriteItemsRequest transaction = TransactWriteItemsRequest.builder()
    .transactItems(
        // 1. Decrement inventory (fail if stock is 0)
        TransactWriteItem.builder().update(
            Update.builder()
                .tableName("shopflow-inventory")
                .key(Map.of("product_id", AttributeValue.fromS("prod-iphone-15")))
                .updateExpression("SET stock_count = stock_count - :qty")
                .conditionExpression("stock_count >= :qty") // fail if insufficient stock
                .expressionAttributeValues(Map.of(":qty", AttributeValue.fromN("1")))
                .build()).build(),
        // 2. Create order record
        TransactWriteItem.builder().put(
            Put.builder()
                .tableName("shopflow-order-history")
                .item(Map.of(
                    "user_id", AttributeValue.fromS("user-123"),
                    "order_id", AttributeValue.fromS("order-456"),
                    "status", AttributeValue.fromS("confirmed"),
                    "created_at", AttributeValue.fromS(Instant.now().toString())
                )).build()).build(),
        // 3. Delete cart item
        TransactWriteItem.builder().delete(
            Delete.builder()
                .tableName("shopflow-cart")
                .key(Map.of(
                    "user_id", AttributeValue.fromS("user-123"),
                    "product_id", AttributeValue.fromS("prod-iphone-15")
                )).build()).build()
    ).build();
dynamoDbClient.transactWriteItems(transaction);
```

07-10

Single-Table Design — ShopFlow Example

Single-table design is the advanced DynamoDB pattern where related entities (Users, Orders, CartItems, Products) are stored in ONE DynamoDB table using generic partition key (PK) and sort key (SK) attributes. It eliminates cross-table limitations and allows retrieving multiple entity types in a single Query. This is the pattern AWS recommends for most DynamoDB workloads.

PK (Partition Key)	SK (Sort Key)	Attributes	What This Item Represents
USER#user-123	PROFILE	email, name, phone, address, created_at	User profile for user-123
USER#user-123	ORDER#2024-11-01T14:30:00Z#order-456	total, status, items_count	Order placed by user-123 at that time
USER#user-123	CART#prod-iphone-15	quantity: 2, price: 999.99, added_at	Cart item for user-123
PRODUCT#prod-iphone-15	METADATA	name, category, seller_id, price, rating	Product detail for iPhone 15
PRODUCT#prod-iphone-15	REVIEW#2024-10-15#review-789	rating: 5, text, user_id	A review of the product
STATUS#PENDING	ORDER#2024-11-01T14:30:00Z#order-456	seller_id, total, product_ids	GSI projection: find all pending orders

```
// Query 1: Get user profile + all orders in one call
QueryEnhancedRequest req = QueryEnhancedRequest.builder()
    .queryConditional(
        QueryConditional.sortBeginsWith(k -> k
            .partitionValue("USER#user-123")
            .sortValue("ORDER#") // get all items with SK starting with ORDER#
        )
    ).build();
// Returns: all orders for user-123 (sorted by order date due to SK design)
```

DynamoDB PITR continuously backs up your table and lets you restore to any second in the last 35 days with no data loss window. AWS also provides on-demand backups (permanent snapshots). For ShopFlow, PITR is enabled on all production tables; on-demand backups are taken before every major deployment that modifies the DynamoDB data model.

■ Enable PITR and Create On-Demand Backup

- 1. DynamoDB → Tables → shopflow-cart → Backups tab
- 2. Point-in-time recovery: click Enable → Enable PITR → takes effect immediately
- 3. Earliest restorable time: shows how far back you can restore (up to 35 days)
- 4. Latest restorable time: typically within the last 5 minutes
- 5. On-demand backup: Create backup → name: shopflow-cart-pre-migration-2024-11-15
- 6. On-demand backups are kept until you delete them; no charge for storage during creation
- 7. To restore: Backups tab → select backup → Restore table → new table name
- 8. Restoring creates a new table; you then copy missing data from the restored table to the live table

Recovery Method	Recovery Point	Recovery Time	Downtime	Best For
PITR restore	Any second in last 35 days	10-30 minutes (creates new table)	Zero (restores to new table)	Accidental deletion of production data
On-demand backup restore	Exact snapshot point	10-30 minutes	Zero	Pre-deployment checkpoint; compliance
AWS Backup (cross-region)	Daily backup, copied to secondary region	30-60 minutes	Zero	DR: recover DynamoDB data in a different region

PartiQL is a SQL-compatible query language for DynamoDB. Instead of writing DynamoDB API calls (QueryEnhancedRequest, expressionAttributeValue, etc.), you can write familiar SQL syntax. PartiQL is useful for: ad-hoc queries in the console, data exploration, one-time migrations, and simple CRUD operations. It does NOT support JOINS or complex aggregations — still need to design proper primary keys.

■ Run PartiQL Query in DynamoDB Console

- 1. DynamoDB → PartiQL editor (left sidebar)
- 2. Run a SELECT: SELECT * FROM "shopflow-cart" WHERE user_id = "user-123"
- 3. Click Run — results appear in the table below
- 4. Run an UPDATE: UPDATE "shopflow-cart" SET quantity = 3 WHERE user_id = "user-123" AND product_id = "prod-iphone-15"
- 5. Run a DELETE: DELETE FROM "shopflow-cart" WHERE user_id = "user-123" AND product_id = "prod-test"
- 6. Note: SELECT without WHERE scans the entire table (expensive for large tables!)

```
// Java: Execute PartiQL via DynamoDB SDK
ExecuteStatementRequest statementRequest = ExecuteStatementRequest.builder()
    .statement(
        "SELECT * FROM \"shopflow-cart\" WHERE user_id = ?"
    )
    .parameters(AttributeValue.fromS("user-123"))
    .build();
ExecuteStatementResponse response = dynamoDbClient.executeStatement(statementRequest);
```

```
List> items = response.items();
```

■ Use PartiQL for ad-hoc console exploration and one-time data fixes, but use the DynamoDB SDK API (Query, PutItem, etc.) in production application code. SDK APIs are more explicit about capacity consumption and access patterns, which helps with cost optimization and debugging.

DynamoDB Accelerator (DAX) — Microsecond Caching

DAX (DynamoDB Accelerator) is a fully managed, in-memory cache specifically for DynamoDB. It is API-compatible: your application code does not change (just swap the DynamoDB client for the DAX client). DAX reduces read latency from 1-3ms to microseconds for frequently read items. ShopFlow uses DAX for the product catalog table — product details are read thousands of times per second but rarely change.

■ DAX vs ElastiCache for DynamoDB

DAX is DynamoDB-specific: it understands DynamoDB items, caches at the item level, handles cache invalidation automatically when items are updated, and is API-compatible (no code change). ElastiCache (Redis) is generic: you cache arbitrary data, manage keys yourself, and must manually invalidate when DynamoDB data changes. For DynamoDB-specific caching, DAX is simpler. For cross-service caching (cache both DynamoDB and external API responses), ElastiCache is more flexible.

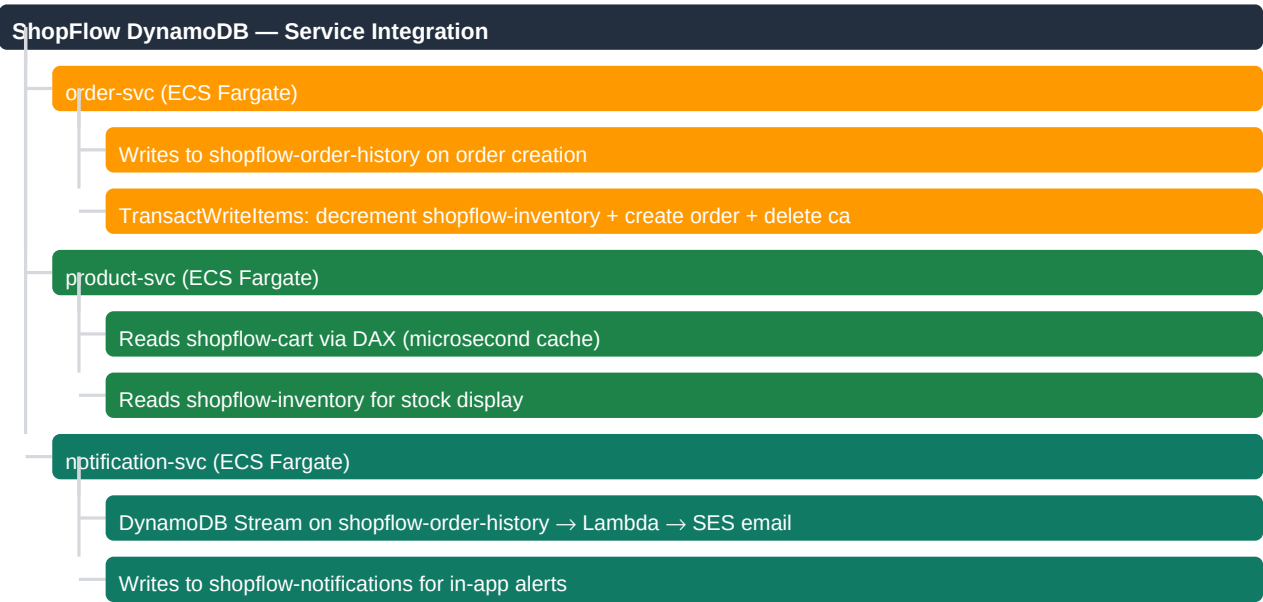
Example: ShopFlow product-svc: swaps DynamoDB client for DAX client. GET /api/products/iphone-15 → DAX checks cache (hit: 50µs response, zero DynamoDB read cost) → (miss: DAX fetches from DynamoDB, caches for 5 minutes, returns 2ms response).

DAX Configuration	Value	Reasoning
Cluster size	3 nodes (1 per AZ) for HA	Each node is an r6g.large; DAX auto-replicates across nodes; if one AZ fails
Item TTL	300 seconds (5 minutes)	Product details change infrequently; 5-minute stale risk acceptable vs cost of
Query TTL	60 seconds (1 minute)	Query results (product list by category) change more often (new products ad
Write-through	Enabled	When product-svc updates a product, DAX updates both DynamoDB and the

ShopFlow DynamoDB Architecture — All Tables

Here is the complete DynamoDB architecture for ShopFlow — all tables, their access patterns, capacity modes, and how they integrate with the microservices.

Table	Partition Key	Sort Key	GSIs	Capacity	Used By
shopflow-cart	user_id	product_id	product-carts-index (PK=product_id)	On-Demand	product-svc reads; order-svc writes
shopflow-order-history	user_id	order_id (ORDER#date#id)	status-created-index	On-Demand	order-svc writes; user-svc reads
shopflow-inventory	product_id	location_id	low-stock-index (PK=warehouse_id)	Provisioned (10K WCU)	inventory-svc: atomic decrements
shopflow-sessions	session_id	—	user-sessions-index (PK=user_id)	On-Demand + TTL=24h	user-svc: validate JWT tokens
shopflow-rate-limits	api_key#endpoint	window (YYYY-MM-DD-HH-MM)	—	On-Demand + TTL=window	API Gateway Lambda authorizer
shopflow-notifications	user_id	notification_id (date#id)	status-index (PK=status)	On-Demand + TTL=30 days	notification-svc writes; user-svc reads



07-15

Monitoring DynamoDB — CloudWatch Metrics

DynamoDB publishes metrics to CloudWatch that help you detect: throttling (requests being rejected because capacity is exceeded), latency (slow queries that may indicate table scan or missing GSI), and errors (failed transactions). ShopFlow monitors these in a CloudWatch dashboard and alerts on-call engineers when throttling exceeds 1% of requests.

Metric	Alert Threshold	Meaning If Triggered	Action
ConsumedReadCapacityUnits	> 80% of provisioned RCU for 5 min	Table is near read capacity limit; throttling may start	Increase provisioned RCU or switch to on-demand
ThrottledRequests	> 0 for any 1-minute period	Some reads or writes are being rejected; application is retrying	Check access patterns for Scans or large reads
SuccessfulRequestLatency	P99 > 10ms for GetItem or Query	Unusually high latency; may indicate hot partition	Check for hot partitions or large reads
SystemErrors	> 0	Internal DynamoDB error (rare)	Retry with exponential backoff; if persistent, contact AWS
TransactionConflicts	> 100/min	Multiple transactions competing for the same item	Review transaction access patterns

07-16

Hands-On Lab — Build ShopFlow Cart Feature

This lab implements the complete ShopFlow shopping cart using DynamoDB. You will create the table, write a Spring Boot REST API, test all cart operations, and observe TTL deletion. Estimated time: 30 minutes.

Step	Action	Verification
1	Create shopflow-cart table (console: PK=user_id, SK=product_id, TTL=shopflow-cart → Status: Active)	DynamoDB console → shopflow-cart → Status: Active
2	Add DynamoDB dependency to pom.xml: software.amazon.awssdk:dynamodb:2.10.0 (dynamodb-enhanced)	aws-sdk-java-core:2.10.0, dynamodb-enhanced
3	Create CartItem.java with @DynamoDbBean annotations (from 07-04)	CartItem.java compiled without errors
4	Create CartRepository.java with getCart, addItem, removeItem methods	Unit tests pass
5	Create CartController.java: GET /cart/{userId}, POST /cart/{userId}/items/{productId}	Spring Boot REST API up and running
6	Test: POST /cart/user-001/items {productId: "prod-abc", quantity: 1}	HTTP 201; item visible in DynamoDB console
7	Test: GET /cart/user-001	HTTP 200; JSON array with 1 item
8	Set TTL to 1 minute in the past: expires_at = now - 60 seconds	DynamoDB console → item shows expires_at; wait up to 48hr for TTL deletion

Step	Action	Verification
9	Add GSI: product-carts-index (PK=product_id) via console — Click here to create index	Click here to verify

07-17

DynamoDB Export to S3 — Data Lake Integration

DynamoDB can export the full contents of a table to S3 in DynamoDB JSON or Amazon Ion format, without consuming table read capacity and without impacting table performance. ShopFlow exports the order-history table nightly to S3 for Athena analytics queries — the export is read by the analytics pipeline without touching the production DynamoDB table.

■ Export DynamoDB Table to S3

1. DynamoDB → Tables → shopflow-order-history → Exports and streams tab
2. Click Export to S3
3. Destination S3 bucket: shopflow-analytics-data-prod
4. S3 key prefix: ddb-exports/order-history/
5. Export from time: Latest (point-in-time snapshot using PITR; PITR must be enabled)
6. Export format: DynamoDB JSON (or Amazon Ion for Glue/EMR)
7. Click Export — export runs in background; does not consume table RCUs
8. Monitor: Exports tab shows progress → Completed in 5-15 min for large tables
9. Query with Athena: CREATE EXTERNAL TABLE order_history_export LOCATION s3://shopflow-analytics-data-prod/ddb-exports/order-history/

Export Feature	Detail	ShopFlow Benefit
Zero table impact	Export reads from PITR snapshots; does not use table RCUs	Highly scalable item export does not slow down production order reads
Point-in-time	Export from any time in PITR window (last 35 days)	Export "as of end of day" for daily analytics snapshots
Format options	DynamoDB JSON: native format; Amazon Ion: compact binary	Use DynamoDB JSON for Athena (automatically handled by Dyn)
Incremental export	Export only items changed since last export (requires PITR)	Daily incremental exports after initial full export: faster and cheaper

07-18

Conflict Resolution — Optimistic Locking

When two processes try to update the same DynamoDB item simultaneously, one update can overwrite the other silently. Optimistic locking prevents this: each item has a version number. When updating, you check that the version is what you expect (condition expression). If another process updated it first, the condition fails — you retry with the new value. ShopFlow uses optimistic locking on inventory counts during flash sales.

```
// Optimistic locking pattern for inventory update
// Step 1: Read current item with version
getItemRequest = GetItemRequest.builder()
    .tableName("shopflow-inventory")
    .key(Map.of("product_id", AttributeValue.fromS("prod-iphone-15")))
    .build();
Map item = dynamoDbClient.getItem(getItemRequest).item();
int currentVersion = Integer.parseInt(item.get("version").n());
int currentStock = Integer.parseInt(item.get("stock_count").n());
// Step 2: Update with condition check on version
try {
    dynamoDbClient.updateItem(UpdateItemRequest.builder()
        .tableName("shopflow-inventory")
```

```

    .key(Map.of("product_id", AttributeValue.fromS("prod-iphone-15")))
    .updateExpression("SET stock_count = :newStock, version = :newVersion")
    .conditionExpression("version = :currentVersion AND stock_count >= :qty")
    .expressionAttributeValues(Map.of(
        ":newStock", AttributeValue.fromN(String.valueOf(currentStock - 1)),
        ":newVersion", AttributeValue.fromN(String.valueOf(currentVersion + 1)),
        ":currentVersion", AttributeValue.fromN(String.valueOf(currentVersion)),
        ":qty", AttributeValue.fromN("1")
    )).build());
} catch (ConditionalCheckFailedException e) {
    // Another process updated; retry from step 1 (up to 3 times)
}

```

07-19

DynamoDB Cost Breakdown and Optimization

DynamoDB cost has three components: read/write capacity units consumed, storage, and optional features (DAX, Streams, PITR). At ShopFlow scale, these are the primary costs and optimizations:

Cost Item	Volume	Monthly Cost	Optimization
shopflow-inventory writes (On-Demand)	10M writes/month (flash sales)	\$12.50/month	Switch to Provisioned: 400 WCU baseline → \$18/month
shopflow-order-history reads (GSI)	50M reads/month	\$12.50/month	Enable DAX: 80% reads served from cache → only \$2.50/month
DAX cluster (3 nodes r6g.large)	730 hours × 3 × \$0.269/hr	\$590/month	Worth it if DynamoDB read cost saved > \$590/month
DynamoDB Streams	Enabled on order-history (10M changes/month) × 10MB/min → \$24/month	\$24/month	Worth the Lambda integration for real-time analytics
PITR (all 6 tables)	50 GB average across all tables	\$5/month (100% table size)	Keep enabling only for extra reads cost
Storage (all tables combined)	100 GB	\$25/month	Enable TTL on all temporary data tables (cart, session)

07-20

NoSQL Design Anti-Patterns to Avoid

The most common DynamoDB mistakes cost significantly in performance, cost, and reliability. These are the patterns that cause problems in production at scale.

Anti-Pattern	Why It Is Wrong	ShopFlow Correct Pattern
Using email as partition key	Email has low cardinality; creates hot partitions; users will fault if high cardinality	Use user_id (UUID) as high cardinality, distributes evenly across partitions
Scanning to find records	SELECT * from table without PK = full table scan	Design GSI for every 1st ES pattern. If not, create the table with a GSI
Storing large blobs in items	DynamoDB item max: 400 KB; images, videos, PDFs don't fit in 400 KB items	Store binary data in S3; store only the S3 key in DynamoDB
Many small tables (one per entity type)	Forces cross-table reads (no JOINS); limits single-table design of related data	Single table design of related data; use PK/LS
Not enabling PITR	First time an engineer runs DeleteTable in prod = data loss	Enable PITR on all data partitions immediately after creation
Using Scan for analytics	Production analytics Scans slow down the table	Export to S3 nightly, used by analytics; use a separate table for analytics

07-21

DynamoDB with Lambda — Serverless Cart Microservice

DynamoDB pairs naturally with Lambda: both are serverless, both scale to zero, and both charge only for what you use. A Lambda-based cart microservice has zero fixed cost when no users are shopping (nights and weekdays for a B2C app). ShopFlow uses this pattern for the notifications service and analytics processors.

```

// Lambda function: Get cart items for a user
public class GetCartHandler implements RequestHandler {
    private final DynamoDbEnhancedClient client = DynamoDbEnhancedClient.create();
    private final DynamoDbTable cartTable = client
        .table("shopflow-cart", TableSchema.fromBean(CartItem.class));
    @Override
    public APIGatewayProxyResponseEvent handleRequest(

```

```
APIGatewayProxyRequestEvent event, Context context) {
String userId = event.getPathParameters().get("userId");
// Query all cart items for this user
List items = cartTable.query(
QueryEnhancedRequest.builder()
.queryConditional(
QueryConditional.keyEqualTo(k -> k.partitionValue(userId))
).build()
).items().stream().toList();
return new APIGatewayProxyResponseEvent()
.withStatusCode(200)
.withBody(new ObjectMapper().writeValueAsString(items));
}
```

■ Use Lambda SnapStart for Java Lambda functions that call DynamoDB. SnapStart takes a snapshot of the initialized Lambda after the first cold start, then restores from the snapshot for subsequent invocations. This reduces Java cold start from 3-5 seconds to 200-500ms — critical for a shopping cart API that customers notice.

07-22

Inventory Management at Flash Sale Scale

Flash sales are the most challenging DynamoDB pattern: thousands of concurrent users trying to buy the last 100 units of a product. Each purchase must decrement the inventory count atomically. Poor design causes: overselling (selling 150 units when only 100 exist), lost updates (concurrent writes overwrite each other), or hot partitions (all writes to one product_id).

Flash Sale Scenario	Naive Approach (Wrong)	ShopFlow Correct Approach
1,000 users simultaneously try to buy the last 150 units of a product	Read stock (150), write stock - 1. Conditional update fails. Multiple threads write. Stock proceeds > 100 units.	Conditional update. Multiple threads write. Stock proceeds > 100 units.
Product has 10 variants (colors) all on sale	Use one item per SKU: 10 items all update same partition. Inventory items split (all SKUs to product_id). Hot partitions.	Create separate inventory items for all SKUs to product_id. Hot partitions.
Real-time inventory count display	Read shopflow-inventory on every product page. Reads 100K Reads in read-through cache.	Read from Redis in read-through cache.

07-23

DynamoDB Schema Versioning

Unlike Aurora (where Flyway migrations change the schema centrally), DynamoDB has no schema — each item can have different attributes. Schema changes happen gradually: new items use the new schema; old items remain. ShopFlow handles this with a schema_version attribute and application-level migration logic.

```
// DynamoDB schema versioning pattern
@DynamoDbBean
public class CartItem {
// Always-present fields
@DynamoDbPartitionKey String userId;
@DynamoDbSortKey String productId;
int schemaVersion; // 1 for old items; 2 for new items with discountCode
int quantity;
double price;
// New field (only in schema v2 items; null in v1 items)
@DynamoDbAttribute("discount_code")
String discountCode; // null for v1 items; present for v2
}

// Service: handle both v1 and v2 items gracefully
CartItem item = cartTable.getItem(key);
if (item.getSchemaVersion() == null || item.getSchemaVersion() < 2) {
// Old v1 item: no discountCode field
item.setSchemaVersion(2);
item.setDiscountCode(null);
cartTable.putItem(item); // lazy migration: update to v2 on next touch
}
```

Before launching ShopFlow DynamoDB tables to production, run this checklist. Each item prevents a common production issue.

Check	Console Verification	Pass Criteria
PITR enabled on all tables	DynamoDB → Tables → each table → Backups tab	Point-in-time recovery: Enabled on every production table
TTL configured for expirable data	DynamoDB → Tables → shopflow-cart → Additional settings	TTL settable with expires_at attribute; test item with past expires_at
No Scan in application code	Search codebase for .scan()	Zero .scan() calls in production code paths (OK in admin UI)
GSIs cover all query patterns	List all API access patterns; trace each to PK/SK or GSI	Every "Query X" query uses a GSI Query, not a Filter or Scan
On-Demand for spiky tables; Provisioned for steady tables	DynamoDB → Tables → Capacity tab	shopflow-inventory: Provisioned + auto scaling; shopflow-cart: On-Demand
Monitoring alarms set	CloudWatch → Alarms	Alarms on ThrottledRequests > 0, SuccessfulRequestRateLow
DynamoDB Streams enabled for event-driven architectures	Tables → shopflow-tables-history → Exports and streams	Streams: Enabled (New and old images); Lambda triggers on new images
Item size < 10 KB for high-volume tables	DynamoDB → Metrics → ItemSizeBytes (from X-Ray logging)	Average item size < 10 KB; no large embedded documents

DynamoDB Global Tables replicate your table to multiple AWS regions simultaneously. Each region has a full copy with read AND write access. ShopFlow uses Global Tables for the sessions table — a user who logs in from London should have their session available in us-east-1 (where most compute runs) and in eu-west-1 (for low-latency session validation). Writes in any region automatically replicate to all other regions within < 1 second.

Global Tables Feature	Description	ShopFlow Use Case
Active-active writes	Any region can accept writes; all replicated globally	User logs in via EU CloudFront → session written to eu-west-1 and replicated to us-east-1
Last-writer-wins conflict resolution	Multiple regions update the same item simultaneously; the latest timestamp wins	By the latest timestamp (session_id per user per login); accept updates from any region
Regional isolation	Each region can function independently during infrastructure outages	If AWS S3 bucket in us-east-1 goes down, each region serves local users with sessions from local DynamoDB
Replication cost	\$0.25 per million rWCUs (replicated write units)	shopflow-sessions: 10M writes/month × \$0.25/M × 1 additional region = \$2.50/month

■ Create DynamoDB Global Table

1. DynamoDB → Tables → shopflow-sessions → Global Tables tab
2. Click Create replica
3. AWS Region: EU (Ireland) eu-west-1
4. Click Create replica — replication starts; existing data synced to eu-west-1
5. After ~5 minutes: eu-west-1 DynamoDB console shows shopflow-sessions table
6. Update ECS tasks in eu-west-1 to write sessions to local eu-west-1 DynamoDB endpoint
7. Sessions now replicate to us-east-1 automatically

DynamoDB IAM conditions let you restrict application users to accessing only their own data. Using Cognito identity federation, an authenticated user gets temporary AWS credentials that are restricted to only the DynamoDB items where the partition key matches their Cognito user ID. This enforces data isolation at the IAM layer — even if the application code has a bug that passes the wrong user_id, the DynamoDB call is rejected by IAM.

```
// IAM policy attached to Cognito authenticated role
```

```
// Allows a user to read/write ONLY their own cart items
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["dynamodb:GetItem", "dynamodb:PutItem", "dynamodb:UpdateItem",
      "dynamodb>DeleteItem", "dynamodb:Query"],
    "Resource": "arn:aws:dynamodb:us-east-1*:table/shopflow-cart",
    "Condition": {
      "ForAllValues:StringEquals": {
        // user_id must match the authenticated Cognito user ID
        "dynamodb:LeadingKeys": ["${cognito-identity.amazonaws.com:sub}"]
      }
    }
  }]
}
// Result: user "abc-123" can ONLY access cart items where user_id = "abc-123"
// GetItem with user_id = "other-user-456" returns AccessDeniedException
// This prevents horizontal privilege escalation even if frontend has a bug
```

■ Defense in Depth for DynamoDB

Implement access control at THREE layers: (1) Application layer: order-svc validates user_id from JWT before building DynamoDB key. (2) IAM layer: Cognito role conditions restrict DynamoDB LeadingKeys to the authenticated user's sub. (3) DynamoDB: item-level access via fine-grained IAM conditions. An attacker who bypasses the application layer is still stopped by IAM; a misconfigured IAM policy is caught by application validation.

07-27

DynamoDB Capacity Planning for Black Friday

ShopFlow expects 10x normal traffic on Black Friday. The inventory table (shopflow-inventory) is the most write-intensive: every purchase decrements a counter. Planning the right capacity prevents throttling during the sale.

Metric	Normal Day	Black Friday Peak	Provisioned Capacity (+ buffer)
shopflow-inventory WCU	5,000 writes/min = 84 WCU	50,000 writes/min = 834 WCU	1,200 WCU (44% buffer above peak)
shopflow-inventory RCU	20,000 reads/min = 334 RCU	30,000 reads/min = 500 RCU	800 RCU (60% buffer)
shopflow-order-history WCU	2,000 orders/min = 34 WCU	20,000 orders/min = 334 WCU	On-Demand (unpredictable spike; no throttle)
shopflow-cart WCU	8,000 cart changes/min = 134 WCU	80,000 cart changes/min = 1,334 WCU	On-Demand (traffic highly variable during flash sale)

■ Update DynamoDB Capacity Before Black Friday

1. DynamoDB → Tables → shopflow-inventory → Capacity tab
2. Current: Provisioned 100 WCU / 400 RCU
3. Click Manage capacity
4. Change to: 1,200 WCU write capacity / 600 RCU read capacity
5. Auto scaling: enabled; min=1200, max=2000 (allow further burst)
6. Apply immediately (not during maintenance window — capacity changes take effect in minutes)
7. Set CloudWatch alarm: ThrottledRequests > 0 → SNS → PagerDuty alert
8. After Black Friday (Saturday afternoon): reduce back to 100/400 to save cost

07-BP

Best Practices and Anti-Patterns

✓ Design access patterns first, then design the DynamoDB table — unlike SQL where you can always add queries later, DynamoDB requires you to know how you will access data before creating the table. Write down all your access patterns (get cart by user, get orders by status, etc.) before touching the console

✓ Use single-table design for related entities — storing users, orders, and cart items in one table (using item type prefixes like USER#123, ORDER#456) avoids cross-table limitations and reduces costs. One GetItem or Query returns all related data in one request

✓ Enable point-in-time recovery on all production tables — PITR is free (included in table pricing) and lets you restore the table to any second in the last 35 days. No downtime to enable or use

✓ Use TTL for automatic cleanup of temporary data — expired sessions, rate limit counters, temporary locks. TTL deletion is free and prevents unbounded table growth

✓ Use On-Demand capacity for new tables and development — switch to Provisioned + Auto Scaling only after you have 2+ weeks of CloudWatch metrics showing predictable traffic patterns

■ Never use Scan on large production tables — Scan reads every item in the table; at 1M items it reads 1M items (costs 1M RCUs) even if you want only 1 result. Always use Query (with a partition key) or GetItem (with the full primary key). Design GSIs to avoid Scans

■ Do not use DynamoDB for complex relational queries — if you find yourself needing 5+ GSIs to support your access patterns, or doing multiple GetItem calls and joining results in application code, Aurora PostgreSQL is the better choice for that data

07-QR

Quick Reference and Interview Q&A

Question	Answer
What is a hot partition and how do you avoid it?	A hot partition occurs when too many requests go to the same DynamoDB partition (determined by partition key).
What is the difference between GSI and LSI?	LSI (Local Secondary Index): same partition key as base table; different sort key. Created at table creation.
What is DynamoDB adaptive capacity?	When one partition becomes hot (receives more traffic than its allocated capacity), adaptive capacity automatically increases the capacity of that partition.